Project 1

- **Description:**
  -
    - Question
      1. Requirement:
         - In this project, you need to write a new system call `void * my_get_physical_addresses(void *)` so that a process can use it to get the physical address of a virtual address of a process.
         - The return value of this system call is either 0 or an address value. 0 means that there is no physical address assigned to the logical address currently. A non-zero value means the physical address of the logical address submitted to the system call as its parameter (in fact, this address is only the offset of a logical address).
      2.
         ```
         //prototype of the new system call is as follows:

         void * my_get_physical_addresses(void *)
         ```
      3. **Question 1** (**50** points)
         What follows is an example code which you can use to see the effect of copy-on-write.

         ```
         #include <stdio.h>


         //void * my_get_physical_addresses(void *);


         int global_a=123;  //global variable

         void hello(void)
         {
            printf("=======================================================================================================
         }


         int main()
         {
           int      loc_a;
           void     *parent_use, *child_use;

           printf("===========================Before Fork==================================\n");
           parent_use=my_get_physical_addresses(&global_a);
           printf("pid=%d: global variable global_a:\n", getpid());
           prinft("Offest of logical address:[%p]   Physical address:[%p]\n", &global_a,parent_use);
           printf("========================================================================\n");


           if(fork())
           { /*parent code*/
             printf("vvvvvvvvvvvvvvvvvvvvvvvvvv  After Fork by parent  vvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvv\n");
             parent_use=my_get_physical_addresses(&global_a);
             printf("pid=%d: global variable global_a:\n", getpid());
             prinft("******* Offset of logical address:[%p]   Physical address:[%p]\n", &global_a,parent_use);
             printf("vvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvv\n");
             wait();
           }
           else
           { /*child code*/

             printf("llllllllllllllllllllllllll  After Fork by child  llllllllllllllllllllllllllllllll\n");
             child_use=my_get_physical_addresses(&global_a);
             printf("******* pid=%d: global variable global_a:\n", getpid());
             printf("******* Offset of logical address:[%p]   Physical address:[%p]\n", &global_a, child_use);
             printf("lllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllll\n");
             printf("____________________________________________________________________________\n");

             /*----------------------- trigger CoW (Copy on Write) -----------------------------------*/
             global_a=789;

             printf("iiiiiiiiiiiiiiiiiiiiiiiiii  Test copy on write in child  iiiiiiiiiiiiiiiiiiiiiiii\n");
             child_use=my_get_physical_addresses(&global_a);
             printf("******* pid=%d: global variable global_a:\n", getpid());
             prinft("******* Offset of logical address:[%p]   Physical address:[%p]\n", &global_a, child_use);
             printf("iiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiiii\n");
             printf("____________________________________________________________________________\n");
             sleep(1000);
           }
         }
         ```
      4. **Question 2** (**50** points)
         What follows is an example code which you can use to check whether a loader loads all data of a process before executing it.

         ```
         #include <stdio.h>


         //void * my_get_physical_addresses(void *);
         ```

```
int a[2000000];


int main()
{
  int      loc_a;
  void     *phy_add;


  phy_add=my_get_physical_addresses(&a[0]);
  printf("global element a[0]:\n", );
  prinft("Offest of logical address:[%p]   Physical address:[%p]\n", &a[0], phy_add);
  printf("========================================================================\n");
  phy_add=my_get_physical_addresses(&a[1999999]);
  printf("global element a[1999999]:\n", );
  prinft("Offest of logical address:[%p]   Physical address:[%p]\n", &a[1999999], phy_add);
  printf("========================================================================\n");


}
```

5. Hint:
   - Two threads show a physical memory cell (one byte) if both of them have a virtual address that is translated into the physical address of the memory cell.
   - The kernel usually does not allocate physical memories to store all code and data of a process when the process starts execution.
   - Inside the Linux kernel, you need to use function `copy_from_user()` and function `copy_to_user()` to copy data from/to a user address buffer.
   - Check the "Referenced Material" part of the Course web site to see how to add a new system call in Linux.

- **Project Submission:**
  - Due time: **23:55 10th Nov.**
  - The demo will be held from **11th Nov.** to **12 Nov.**
  - Please fill out this form to choose your demo time before **10th Nov.**
  - On site demo of this project is required.
  - During on site demo, the TAs will execute several programs written by them to check the correctness of your system calls.
  - When demonstrating your projects, the TAs will ask you some questions regarding to your projects. Part of your project grade is determined by your answers to the questions.
  - NEW Report Content:
    - Do not forget writing the names and student IDs of all members in your team.
    - Your report should be in **hackmd** document form and contain:
      - Your source code
      - the execution results
    - Submit the URL of your **hackmd** document to the new-eeclass.
  - Late submission will **NOT** be accepted.